Demystifying the scheduler SLURM Advanced users

Durham HPC days 2026



graphics by ChatGPT

https://www.arc.ox.ac.uk

https://www.it.ox.ac.uk

Contents

1. What problem are we trying to solve?
2. Priority and Fairshare
3. Scheduling mechanisms
4. Resource Locality / Resource Binding
5. Resource Limits
6. Quality of Service
7. There is more...
8. What the scheduler isn't

What do we mean by ”the” scheduler?

An application for facilitating unattended execution of workload.

- Allocate access to resources for some duration of time.
- Arbitrate contention for resources.
- Offer an interface to help users define and submit workload.

In the HPC context, generally refers to cluster managers/schedulers.

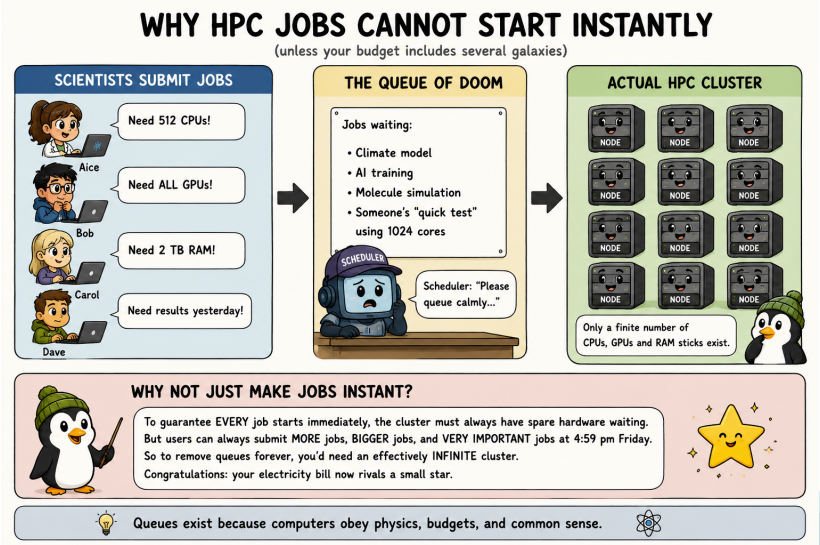
- Grid Engine
- Platform LSF
- PBS
- SLURM
- …

What problem are we trying to solve?

There is more work than there are resources.
We need to prioritise work, taking into account

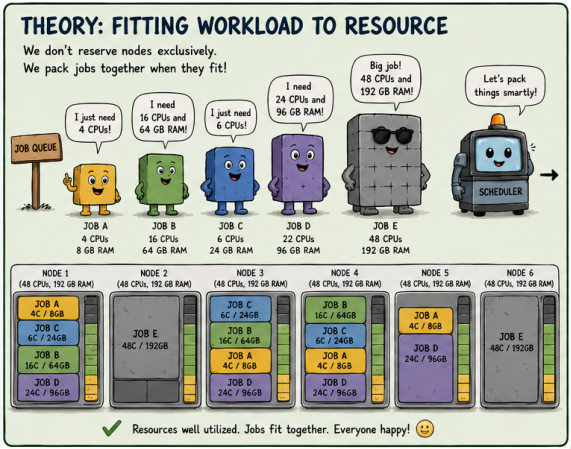
- finite resources,
- competing work,
- optimise throughput,
- optimise latency,
- fairness,
- competing priorities.

Why do I have to wait?



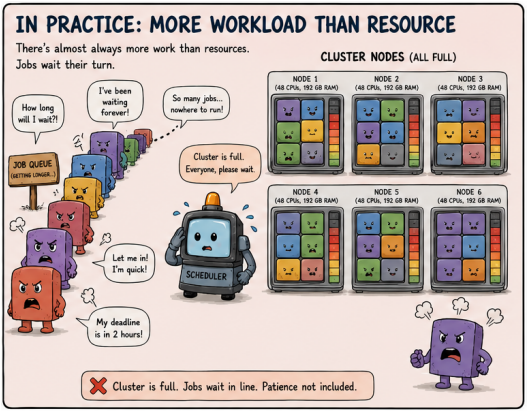
A job scheduler:

- Allows to queue jobs.
- Ensures required resources get allocated.
- Ensures all jobs get the resources they need.
- Enforces limits and policies.
- Ensures resources are shared fairly.



And, most importantly, it tries to ensures resources are as fully utilised.

Priority and Fairshare



The scheduler takes in all queued jobs' requirements, and maps them to the available resource the best way possible.

"Best way possible" has various definitions (and is highly configurable).

Core concept is to maximise resource utilisation whilst sharing them out fairly between all jobs.

..."fair" also has various definitions (and is configurable).

A fundamental concept to how SLURM organises workload is priority.

Multifactor Priority Plugin

There are two plugins to control how SLURM prioritises jobs:

- priority/basic — jobs are prioritised by order of arrival
- priority/multifactor — jobs are prioritised based a range of factors

Overall, jobs are prioritised in the following order:

1. Jobs that can pre-empt.
2. Jobs with an advanced reservation.
3. Partition Priority.
4. Job Priority.
5. Job submit time.
6. Job ID.

The job with the highest job priority is not necessarily the first to be evaluated by the scheduler.

Multifactor Priority Plugin

With ”priority/multifactor”, jobs are prioritized based upon

- age — length of time in the queue
- association — association priority factor
- fairshare — difference between promised and consumed resources
- job size — number of nodes or CPUs a job is allocated
- nice value — factor controlled by users to prioritize their own jobs
- partition — factor associated with each partition
- QOS — factor associated with each Quality Of Service
- site — factor set by an administrator or job_submit/site_factor plugin
- TRES — different for each TRES type, scales with used resource

Job priority is the weighted sum of those factors.

The higher the number, the higher the priority.

Different clusters can be configured to prioritise jobs very differently.

Fair Share

The fairshare factor in the priority/multifactor plugin defaults to the ”Fair Tree” algorithm.

Fair Tree prioritizes such that

- If two accounts — A and B — are siblings, and
- A has a higher fair share factor;
- Then all children of A will have a higher fairshare factor than all children of B.

In other words, all users from a higher priority account receive a higher fair share factor than all users from a lower priority account.

Fair Share

Usage and Shares are the two components of the fairshare factor.

Shares are assigned to associations:

- Representing its nominal ”share” of the system.
- Normalized to 0.0 ...1.0
- Usage represents the accounts proportional usage of the system.
- If Shares == Usage, you have hit your fairshare target.

The algorithm use to determine the fair share factor is (again) highly configurable.

It is fairly common for all associations to have the same share. In practice, that means that the scheduler will try to give everyone the same ”share” of cluster time.

An example (fully made up)

Suppose a cluster is configured with the following factors and weights:

```
PriorityWeightAge=1000
PriorityWeightFairshare=10000
PriorityWeightJobSize=500
PriorityWeightPartition=2000
PriorityWeightQOS=5000
```

SLURM first calculates a normalised factor for each category, and then multiplies each by its configured weight:

```
Priority =
  (AgeFactor × PriorityWeightAge)
+ (FairshareFactor × PriorityWeightFairshare)
+ (JobSizeFactor × PriorityWeightJobSize)
+ (PartitionFactor × PriorityWeightPartition)
+ (QOSFactor × PriorityWeightQOS)
```

An example (fully made up)

Assume two jobs — A and B – have with the following factors:

Job A

Age 0.60
Fairshare 0.25
JobSize 0.80
Partition 1.00
QOS 0.50

A's priority comes to:

```
0.60 × 1000 + 0.25 × 10000
+ 0.80 × 500 + 1.00 × 2000
+ 0.50 × 5000 = 8000
```

Job B

Age 0.9
Fairshare 0.05
Job Size 0.80
Partition 1.00
QoS 0.50

B's priority comes to:

```
0.90 × 1000 + 0.05 × 10000
+ 0.80 × 500 + 1.00 × 2000
+ 0.50 × 5000 = 6300
```

Even though job B has waited longer, job A wins because its fairshare score is much higher and fairshare has the largest weight.

Scheduling mechanisms

SLURM has three scheduling mechanisms:

Direct scheduling Event-driven. Triggers on job submissions; job completions; configuration changes. Low queue depth, quick/simple scheduling. Evaluates jobs in priority order.

Main scheduling loop Runs periodically. Considers all jobs. Evaluates jobs in priority order.

Backfill Most comprehensive scheduling attempt. Attempts to fill the ”gaps” between larger jobs with smaller jobs. Runs periodically.

Backfill will override prioritisation. It considers factors like job run times and resource requirements. If a lower priority job can be started without taking resources from a higher priority job, it will be started.

Backfill improves utilisation without impacting reserved start times.

Scheduling mechanisms

All of these calculations — especially resource projections and backfill — depend on the scheduler knowing what resources are required by the jobs in the queue, and how long they will be required for.

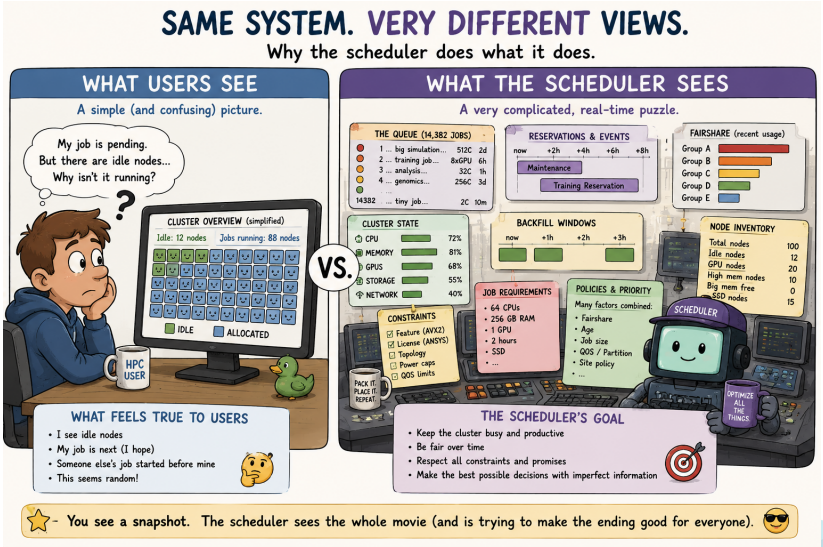
It plans around the time — and resources — requested by the job, not around the ones actually used by the job.

Over-requesting resources (CPUs, memory, run time) negatively impacts the schedulers' ability to plan ahead.

But there are idle [CPUs|GPUs|nodes]!

There are multiple constraints that need to be considered by the scheduler:

- CPU cores
- RAM
- GPUs
- local storage space
- network topology
- run times
- NUMA locality
- reservations
- scheduling policies
- …


SAME SYSTEM. VERY DIFFERENT VIEWS.
Why the scheduler does what it does.
WHAT USERS SEE
A simple (and confusing) picture.
My job is pending.
But there are idle nodes...
Why isn't it running?
CLUSTER OVERVIEW (simplified)
Idle: 12 nodes
Jobs running: 88 nodes
IDLE
ALLOCATED
HPC USER
VS.
WHAT THE SCHEDULER SEES
A very complicated, real-time puzzle.
THE QUEUE (14,382 JOBS)
1 ... big simulation... 512C 2d
2 ... training job... 8xGPU 6h
3 ... analysis... 32C 1h
4 ... genomics... 256C 3d
14382 ... tiny job... 2C 10m
RESERVATIONS & EVENTS
now +2h +4h +6h +8h
Maintenance
Training Reservation
FAIRSHARE (recent usage)
Group A
Group B
Group C
Group D
Group E
CLUSTER STATE
CPU 72%
MEMORY 81%
GPUS 68%
STORAGE 55%
NETWORK 40%
BACKFILL WINDOWS
now +1h +2h +3h
NODE INVENTORY
Total nodes 100
Idle nodes 12
GPU nodes 20
High mem nodes 10
Big mem free 0
SSD nodes 15
CONSTRAINTS
Feature (AVX2)
License (ANSYS)
Topology
Power caps
QOS limits
JOB REQUIREMENTS
64 CPUs
256 GB RAM
1 GPU
2 hours
SSD
...
POLICIES & PRIORITY
Many factors combined:
Fairshare
Age
Job size
QOS / Partition
Site policy
...
SCHEDULER
PACK IT. PLACE IT. REPEAT.
OPTIMIZE ALL THE THINGS
WHAT FEELS TRUE TO USERS
I see idle nodes
My job is next (I hope)
Someone else's job started before mine
This seems random!
THE SCHEDULER'S GOAL
Keep the cluster busy and productive
Be fair over time
Respect all constraints and promises
Make the best possible decisions with imperfect information
You see a snapshot. The scheduler sees the whole movie (and is trying to make the ending good for everyone).

CPU Binding

Depending on configuration, SLURM will automatically bind tasks to appropriate CPU resources. There are a range of options to control the binding behaviour. These can be configured globally; per partition; per node; or modified on job submission.

Order of enforcement of binding options is:

1. srun options
2. node configuration
3. partition configuration
4. global configuration

SLURM sets a number of environment variables (all starting SLURM_CPU_BIND) in the job environment that contain information about what the binding is.

CPU Binding

Some of the options available to ”`--cpu-bind`” are:

none Disable binding.

verbose Report binding before each task run.

rank If using an entire node, bind by task rank.

rank_ldom If using an entire node, bind by NUMA locality by rank.

Again, see the ”srun” man page for details.

Generic RESources (GRES)

SLURM has functionality that allows to define, and schedule, arbitrary resources. What, and how, and how they are controlled, is specific to each resource.

GRES resources are defined and controlled by plugins. They are a node-level resource.

The most common resources scheduled through this are GPUs. However, there are other examples (NICs/bandwidth). There is a ”generic” plugin, allowing sites to add their own.

Slurm’s job scheduler heeds affinity when scheduling GRES resources — unless disabled by configuration or at job submission. GRES affinity is handles on a socket basis internally. It does not respect core-level affinity during job scheduling.

Network Topology

SLURM can be configured for topology aware resource allocation through topology plugins.

A very common configuration is a hierarchical network (”tree” topology). The basic idea is that this allows the scheduler to identify the lowest level switch that could satisfy a job requirement, and then allocates resources on this (and it’s underlying leaf switches, if applicable).

It does not ”enforce” an optimal assignment though, generally.

Users can request a maximum number of leaf switches for a job, as well as a maximum time willing to wait for an optimal assignment (”`--switches=count[@time]`” option to sbatch/srun/salloc).

Topology information is not currently autodetected — this is a configuration set by administrators per cluster.

Network Topology

Topology plugins other than ”tree” are available; for example

block Allocate within strictly enforced hierarchical ”blocks”.

ring Models the cluster as ordered rings of nodes.

3D Torus Treats the cluster as a number of 3D torus networks. Jobs are allocated contiguous sub-cubes of nodes within a torus.

For each of these there are again options for users to specify how the placement should be handled.

Resource Limits

Limit can be set on a number of objects – partitions, jobs, associations (users, accounts)…– limiting things like the total number of job that can be run at a time; or submitted at a time; or they can be limits on other resources (GPUs, memory, …); or limits on job wall times.

In general, they are enforced in order:

1. Partition QOS limit.
2. Job QOS limit.
3. User association.
4. Account association(s).
5. Root/Cluster association.
6. Partition limit
7. None

Quality of Service

SLURM has a concept of ”Quality of Service” (QOS) — a QOS can be specified for each job submitted.

A QOS can affect three key job components:

- scheduling priority
- preemption
- resource limits

Access to a QOS is generally set in the user/account association.

Other than setting things like priority and resource limits, a QOS can also be used to override or enforce specific settings — for example, a QOS can be used to allow specific jobs to ignore the time limit on a partition.

There is more...

The concepts included in this talk not an exhaustive list of things SLURM.

For example, there was no mention of checkpointing; preemption; job accounting; job arrays; heterogeneous jobs; job dependencies; containers; the REST API; license management; namespaces; data movement; prologs; epilogs; burst buffers; submission verifiers; multi-cluster operation, …

Nor details of the allocation process — information flows, jobs vs job steps, differences between sbatch / srun / salloc…

We did not even mention multi-node jobs and MPI.

We also didn't cover any scheduler other than SLURM.

The scheduler is not...

- Magic. It will not automatically make your code
  - fast
  - efficient
  - multi-thread, or multi-node capable
- Biased.
- Sentient. It needs to be told things like
  - What resources your code needs;
  - Where it is located;
  - How long it will run for;
  - ...

SLURM also does not have any built-in support for workflows, or data-aware scheduling features.

It is a very complex piece of software, trying to juggle a large amount of variables and competing requirements to come up with the most efficient solution.

Why do HPC admins hate interactive work?

